

ABSTRACT

Campus navigation remains a persistent challenge for new students, faculty, and visitors of Eastern International University (EIU), whose sprawling grounds contain multiple academic blocks, support facilities, parking areas, and pedestrian walkways that are insufficiently represented by commercial mapping platforms.

This report documents the design, implementation of a lightweight cross-platform mobile navigation application built with React Native and the Expo framework. The application renders a fully interactive Google Maps base layer augmented with custom spatial overlays derived from EIU's internal GIS data, originally encoded as Shapefile layers[8] and subsequently converted into the GeoJSON format[4]. The routing core employs the A* (A-star) pathfinding algorithm operating over a custom-built graph data structure whose nodes and edges are derived from the campus pedestrian road network. The Haversine formula is used as the heuristic for geographic distance estimation.

The application provides point-to-point route planning between labelled landmarks, real-time GPS localisation via the device's native location services, and a polished bottom-sheet user interface.

ACKNOWLEDGEMENT

We would like to express our sincere gratitude to everyone who has supported and encouraged us throughout the development of this project.

First and foremost, we are deeply grateful to our supervisor, Mr. Do Dang Thanh Huy, for his invaluable guidance, continuous support, and dedicated mentorship. His expert advice, constructive feedback, and constant encouragement have been essential in helping us navigate through various challenges and successfully complete this report.

We would also like to extend our thanks to the School of Computing and Information Technology, Eastern International University, for providing a highly functional learning environment and the necessary resources for our research and development.

TABLE OF CONTENTS

ABSTRACT	i
ACKNOWLEDGEMENT	ii
TABLE OF CONTENTS	iii
LIST OF FIGURES.....	iv
LIST OF TABLES.....	v
LIST OF ABBREVIATIONS	vi
CHAPTER 1. INTRODUCTION.....	1
1.1. Project Statement.....	1
1.2. Background and Motivation	1
1.3. Objectives and Scope.....	2
CHAPTER 2. RELATED WORKS	4
2.1. Campus Navigation Systems	4
2.2. Cross-Platform Mobile Frameworks	5
CHAPTER 3. PROPOSED APPROACH, ANALYSIS, AND DESIGN	7
3.1. Requirements Analysis	7
3.2. Spatial Data Architecture and Shapefile Processing	9
3.3. Routing Algorithm and Data Flow	11
3.4. User Interface (UI) Design	16
CHAPTER 4. RESULTS AND DISCUSSION	19
4.1. Implementation Environment.....	19
4.2. Application Interfaces and Functionalities	19
4.3. Demo Observations and Limitations	26
CHAPTER 5. CONCLUSION AND FUTURE WORKS	27
5.1. Conclusion.....	27
5.2. Future Works	27
REFERENCES	29

LIST OF FIGURES

Figure 3.1. System Architecture Overview	9
Figure 3.2. GeoJSON Data Pipeline	11
Figure 3.3. Campus Graph Network Representation.....	12
Figure 3.4. A* Algorithm Flowchart.....	13
Figure 3.5. UI Component Hierarchy	18

LIST OF TABLES

Table 2.1. Framework Comparison	6
Table 3.1. Functional Requirements	7
Table 3.2. Non-Functional Requirements	8
Table 3.3. Campus Landmark Coordinates	10
Table 3.4. streets.json Feature Properties	11
Table 4.1. Development Environment Specifications	19

LIST OF ABBREVIATIONS

No.	Term	Meaning
1	API	Application Programming Interface
2	EIU	Eastern International University
3	GeoJSON	Geographic JavaScript Object Notation
4	GIS	Geographic Information System
5	GPS	Global Positioning System
6	JSON	JavaScript Object Notation
7	JSX / TSX	JavaScript XML / TypeScript XML
8	SDK	Software Development Kit
9	UI / UX	User Interface / User Experience
10	WGS-84	World Geodetic System 1984

CHAPTER 1. INTRODUCTION

1.1. Project Statement

Eastern International University occupies a large, multi-block campus in Binh Duong Province, Vietnam. The campus contains numerous academic buildings (Blocks B3, B8, B9, B10, B11), support facilities such as the Library, Canteen, and multiple Parking Zones, all connected by an internal network of pedestrian pathways and roads. New students, prospective applicants, visiting faculty, and delivery personnel frequently encounter difficulty locating specific facilities within this environment.

The problem is compounded by the fact that commercial mapping services such as Google Maps, Apple Maps, and HERE Maps do not capture the fine-grained internal road topology of private campuses. While these platforms accurately represent the surrounding urban street network, they do not possess data about the pedestrian walkways, named internal roads (e.g., "side-b10-gate4", "circle", "road-canteen"), or the precise spatial relationships between academic blocks within the EIU boundary. A user relying on Google Maps alone would receive no navigational guidance once they enter the campus gate.

This project addresses that gap by building a dedicated, lightweight mobile application that provides indoor-grade navigation for the EIU campus using a custom GIS dataset. The application renders the campus map on top of a standard satellite/street base layer, overlays custom polygon and polyline layers derived from the university's own Shapefile data, and computes optimal walking routes between any two named landmarks using a graph-based pathfinding algorithm.

1.2. Background and Motivation

The proliferation of smartphones has made mobile mapping an essential daily tool for billions of users worldwide. Navigation applications have advanced from simple turn-by-turn street guidance to rich, context-aware platforms integrating indoor positioning, augmented reality, and real-time traffic data. Institutions such as universities, hospitals, airports, and shopping centres have increasingly recognised that commercial maps do not meet their internal wayfinding needs, motivating the development of purpose-built campus navigation solutions.

Several technical trends converge to make this project feasible without specialist hardware:

Cross-platform mobile development frameworks such as React Native[9] allow a single TypeScript codebase to compile to both iOS and Android, significantly reducing engineering effort compared to native development in Swift/Objective-C and Kotlin/Java.

GIS-grade spatial data formats, particularly GeoJSON, provide a lightweight, human-readable encoding of geographic geometries (points, polylines, polygons) that can be bundled directly into a mobile application as a static asset — eliminating the need for a backend map tile server.

Graph-based pathfinding algorithms, specifically A* (A-star), are well-established in computer science and offer optimal shortest-path solutions for the scale of a campus road network (typically fewer than 1,000 nodes), allowing computation to occur entirely on-device in under one millisecond.

The motivation for this specific project arose from the direct experience of students and visitors navigating the EIU campus. The campus's architecture — with parallel rows of similarly styled buildings — creates spatial disorientation, and the absence of visible directional signage compounds the problem. A digital navigation tool accessible on a personal smartphone represents the most scalable and low-maintenance solution.

1.3. Objectives and Scope

Objectives:

1. Develop a functional hybrid mobile application that renders the EIU campus boundary and internal road network as interactive map overlays.
2. Implement a graph-based routing engine (A*) over the custom campus road dataset that computes optimal walking paths between any two named landmarks.
3. Provide real-time GPS localisation so that users can navigate from their current position.
4. Design and implement an intuitive, polished user interface featuring a full-screen map, custom building markers, and a draggable route-planning panel.
5. Bundle all spatial data as static application assets to eliminate any dependency on backend infrastructure.

Scope:

The application is geographically restricted to the EIU campus boundary polygon, defined by the coordinate extent approximately between latitudes 11.0498°N – 11.0558°N and longitudes 106.6631°E – 106.6690°E. The road network dataset covers all internal pedestrian walkways and vehicle roads within this boundary for which GIS data was available at the time of development. The twelve named landmarks included are: Block B3, Block B8, Block B9, Block B10, Block B11 (IT), the Library, the Canteen, the Recruitment Office, Parking Zones 1–4.

The scope explicitly excludes: outdoor navigation on public roads beyond the campus boundary, indoor floor-by-floor navigation within buildings, and real-time dynamic features such as event scheduling or crowd density mapping.

CHAPTER 2. RELATED WORKS

2.1. Campus Navigation Systems

Campus navigation systems have been an active area of research and commercial development for over a decade[2]. Several approaches exist in the literature and in production:

Web-Based GIS Portals: Many universities publish static online maps built with ArcGIS Online, Google Maps Platform, or Leaflet.js. These portals present a browser-accessible map with clickable building markers but offer no turn-by-turn routing for pedestrians. Their primary limitation is the requirement for an internet browser and no mobile-native experience.

Dedicated Native Applications: Institutions such as MIT, Stanford, and the National University of Singapore have deployed iOS/Android applications that use the Google Maps SDK or Apple MapKit with custom overlays for internal navigation. These applications typically maintain a backend database of room-level points of interest and integrate with the institution's identity management system. Their limitation is the cost and complexity of maintaining two separate native codebases.

Hybrid Applications (React Native / Flutter): More recent campus navigation projects have adopted cross-platform frameworks. Hybrid Applications (React Native / Flutter): More recent campus navigation projects have adopted cross-platform frameworks. In these systems, bundling a pre-computed routing graph as a local JSON asset, rather than querying a remote routing server over the network, is a best practice that significantly reduces routing latency and enables offline navigation.

Indoor Navigation Research: For multi-storey buildings, GPS signals are unavailable, and researchers have explored Bluetooth Low Energy (BLE) beacon triangulation, Wi-Fi fingerprinting (Mirowski et al., 2013), and visual positioning systems (VPS) using computer vision. These approaches are beyond the scope of this project but represent important directions for future extension.

The approach taken most closely aligns with the static-asset hybrid model: all routing data is pre-processed and bundled with the application, ensuring that navigation functions correctly even without cellular network connectivity — a significant advantage in areas of poor coverage.

2.2. Cross-Platform Mobile Frameworks

The selection of a development framework is a foundational architectural decision that affects developer productivity, application performance, UI fidelity, and long-term maintainability. Three frameworks dominated consideration for this project:

Flutter (Dart, Google, 2018): Flutter compiles to native ARM code and uses its own rendering engine (Skia/Impeller) rather than platform-native UI widgets, producing highly consistent cross-platform appearance. Its map integration ecosystem (`flutter_map`, `google_maps_flutter`) is mature. However, Flutter's adoption of Dart — a language with limited prior exposure in academic curricula focused on JavaScript/TypeScript — was identified as a learning overhead risk for a time-constrained project.

React Native (JavaScript/TypeScript, Meta, 2015): React Native renders native platform UI components through a JavaScript bridge (or the newer JSI architecture), giving applications a genuinely native look and feel. Its ecosystem is extensive: the `react-native-maps` library[6] provides a production-grade wrapper around both Google Maps SDK (Android) and Apple MapKit (iOS). Critically, the development team possessed prior experience in React, making the framework the most productive choice.`

Ionic (Angular/React/Vue, HTML/CSS): Ionic wraps a web view inside a native shell, meaning the map rendering is performed by the browser engine rather than the native map SDK. This imposes significant performance penalties on map tile rendering and gesture handling, making it unsuitable for a map-centric application.

Decision: React Native with the Expo managed workflow was selected for the following reasons:

- TypeScript-first development with strong type safety for geographic coordinate types.
- `react-native-maps` provides direct access to Google Maps SDK with Polygon, Polyline, and Marker primitives needed for the spatial overlay system.`
- Expo's managed workflow simplifies SDK integration (location, linear gradient, router) without requiring ejection to bare React Native.
- `expo-location`[5] provides a unified, permission-managed GPS API across iOS and Android.`

Table 2.1. Framework Comparison

Criterion	React Native + Expo	Flutter	Ionic
Language	TypeScript	Dart	HTML/JS
Map SDK	Google Maps (native)	Google Maps (native)	Leaflet (WebView)
Map Performance	High	High	Medium
GPS Integration	expo-location	geolocator	Capacitor
Team Familiarity	High	Low	Medium
Selected	☒ Yes	—	—

CHAPTER 3. PROPOSED APPROACH, ANALYSIS, AND DESIGN

3.1. Requirements Analysis

Requirements were elicited through direct observation of campus navigation difficulties and analysis of comparable navigation systems.

Table 3.1. Functional Requirements

ID	Requirement	Priority
FR-01	Display a full-screen interactive map centred on the EIU campus	High
FR-02	Render the campus boundary as a semi-transparent polygon overlay	High
FR-03	Display custom marker badges for all 12 named landmarks	High
FR-04	Request and use device GPS to show user's current position	High
FR-05	Allow user to select a start and end location via a picker modal	High
FR-06	Compute the shortest walking route between two selected points	High
FR-07	Render the computed route as a blue polyline on the map	High
FR-08	Auto-zoom the map to fit the computed route	Medium
FR-09	Allow the user to clear the current route	Medium
FR-10	Support tap-on-marker to set destination and auto-route	Medium
FR-11	Search/filter buildings by name in the picker modal	Medium
FR-12	Collapsible/draggable bottom route-planning sheet	Low

Table 3.2. Non-Functional Requirements

ID	Requirement	Target
NFR-01	Route computation latency	< 100 ms on mid-range Android
NFR-02	Map tile rendering frame rate	≥ 30 fps during pan/zoom
NFR-03	Application cold-start time	< 4 seconds
NFR-04	GPS position update frequency	≤ 3 seconds or ≥ 2 m movement
NFR-05	Bundle size (APK)	< 80 MB
NFR-06	Offline routing capability	Full (no internet needed for routing)

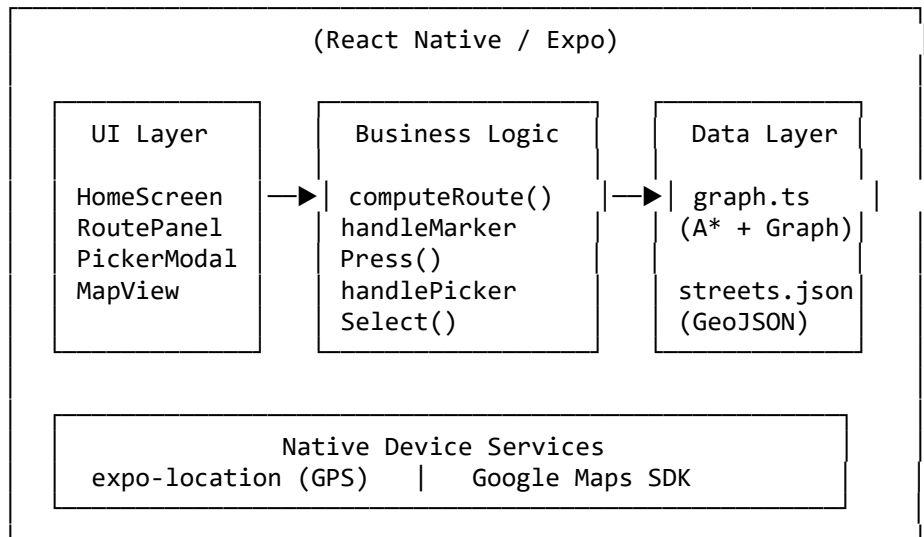


Figure 3.1. System Architecture Overview

3.2. Spatial Data Architecture and Shapefile Processing

3.2.1. Input Data

The spatial dataset for EIU's campus was originally captured in the ESRI Shapefile format[8], an industry-standard binary vector format used by GIS software such as QGIS and ArcGIS. The dataset comprised two primary layers:

1. **Polygon Layer (campus_buildings.json):** Represents the closed boundary polygon of the campus perimeter and the footprints of individual buildings. Encoded as a GeoJSON `MultiPolygon` and `MultiLineString` FeatureCollection with WGS-84 coordinates.

2. **Polyline Layer (streets.json):** Represents the internal pedestrian and vehicle road network as a set of `MultiLineString` features. Each feature includes properties:

- `id` (integer): unique identifier
- `road` (string): Vietnamese road name/tag (e.g., "b8-b9", "gate-gate", "road-canteen")
- `length` (float): pre-computed segment length in metres

Table 3.3. Campus Landmark Coordinates (WGS-84)

ID	Name	Latitude	Longitude
b3	Block 3	11.05251	106.667652
b8	Block 8	11.054710	106.666477
b9	Block 9	11.054120	106.664998
b10	Block 10	11.053615	106.665175
b11	Block 11 (IT)	11.054111	106.666735
canteen	Canteen	11.055053	106.667248
library	Library	11.053667	106.667484
recruit	Recruitment Office	11.052054	106.667976
parking	Parking Zone 1	11.05399	106.667718
parking2	Parking Zone 2	11.05218	106.668420
parking3	Parking Zone 3	11.054900	106.664941
parking4	Parking Zone 4	11.055177	106.666398

3.2.2. Data Conversion Pipeline

The Shapefile-to-application pipeline proceeded as follows:

1. **QGIS Export:** The original `.shp` Shapefile layers were opened in QGIS and exported to GeoJSON format using the "Save Features As" function with the WGS-84 (EPSG:4326) coordinate reference system.
2. **Minification:** The resulting GeoJSON files were minified (whitespace removed) and saved as `assets/streets.json` and `assets/campus_buildings.json`.
3. **Build-Time Bundle:** At build time, the application bundler statically embeds both GeoJSON files directly into the application package. This means that all spatial data is available immediately upon launch without requiring any network connection or external server.

1. **Feature Iteration:** Each GeoJSON Feature whose geometry type is `MultiLineString` is processed. Features with `road` ending in `-in` are skipped.

2. **Coordinate Parsing:** For each LineString within the MultiLineString, consecutive coordinate pairs are processed as directed edges. GeoJSON encodes coordinates as `[longitude, latitude]` (note: reversed from the conventional latitude-first order); this is corrected during parsing.

3. **Node Key Generation:** Each geographic coordinate is converted to a canonical string key by rounding to 5 decimal places (approximately 1-metre precision): `nodeKey(lat, lng) = "${lat.toFixed(5)},${lng.toFixed(5)}"`. This allows nearby-but-identical coordinates from different features to be correctly identified as the same graph node.

4. **Edge Weight:** The edge weight between two adjacent nodes is the Haversine great-circle distance in metres[7], computed by `haversine(lat1, lng1, lat2, lng2)`.

5. **Bidirectional Edges:** Both directions are added (`from→to` and `to→from`), correctly modelling the pedestrian network as an undirected graph.

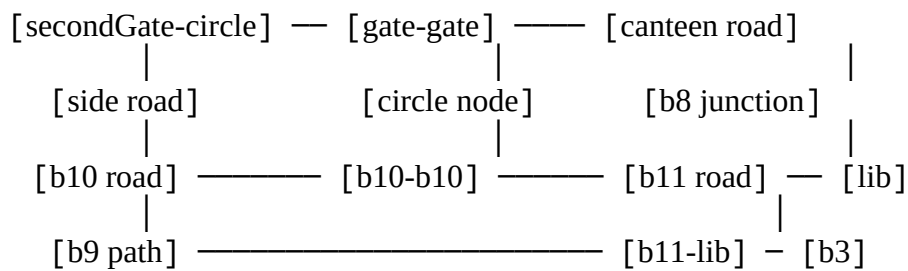


Figure 3.3. Campus Graph Network Representation

(Simplified topological representation — each node represents a coordinate pair at a road intersection or waypoint.)

3.3.2. Snap-to-Graph

Because building entrance coordinates and GPS positions rarely fall exactly on a road network node, a nearest-neighbour search is performed before every route computation. This process scans all available nodes and identifies the one closest to the given coordinate using the Haversine formula. Given the relatively small size of the campus road network, this linear scan is fast enough to be performed in real time without any noticeable delay.

3.3.3. A* Pathfinding

The routing engine implements the classic A* best-first search algorithm[1],[10]. Internally, the algorithm maintains three key data structures: a cost map that tracks the actual distance travelled from the start to each visited node (g-score), an estimated total cost map combining the actual distance with the heuristic distance to the goal (f-score), and a predecessor map used to reconstruct the final path once the destination is reached. A priority queue ordered by f-score ensures that the most promising node is always explored first.

Heuristic $h(n)$: The Haversine distance[7] from node n to the goal — an **admissible** heuristic since the straight-line distance never overestimates the true walking distance on a road network. This guarantees that A* finds the globally optimal (shortest) path.

Algorithm Steps:

1. Initialise: push start node with $f = h(\text{start}, \text{goal})$; $g(\text{start}) = 0$.
2. Loop: pop the node with smallest fScore.
3. If $\text{current} == \text{goal}$: reconstruct path by following `cameFrom` pointers and return the ordered `LatLng[]` array.
4. For each neighbour: compute tentative g score. If better than previously recorded, update g/f scores, record predecessor, add to open set.
5. If open set empties without reaching goal: return `null` (no path exists).

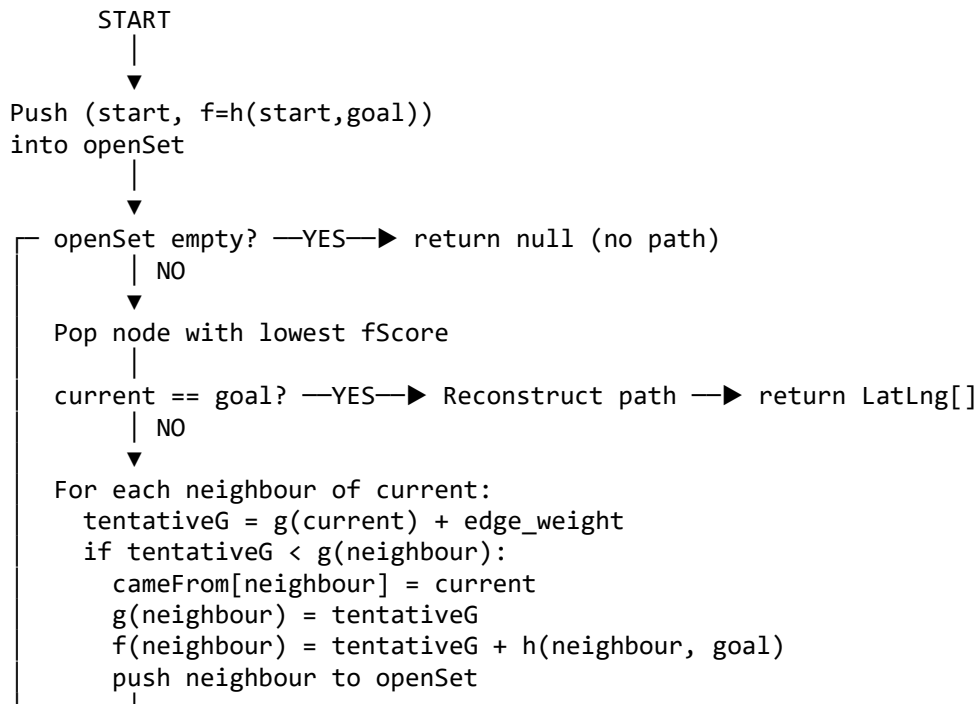


Figure 3.4. A* Algorithm Flowchart

```

function HAVERSINE(lat1, lng1, lat2, lng2) → real
    // Returns the great-circle distance in metres between two GPS coordinates.

function NODE_KEY(lat, lng) → string
    // Returns a canonical string key rounded to 5 decimal places (~1 m precision).
    return format("{lat:.5f},{lng:.5f}")

function SNAP_TO_GRAPH(graph, lat, lng) → nodeKey
    // Returns the key of the graph node nearest to the given GPS coordinate.
    bestKey ← ""
    bestDist ← ∞
    for each (key, coord) in graph.nodes do
        d ← HAVERSINE(lat, lng, coord.lat, coord.lng)
        if d < bestDist then
            bestDist ← d
            bestKey ← key
    return bestKey
function ASTAR(graph, startKey, endKey) → path | null

    // — Edge case



---


    if startKey = endKey then
        return [ graph.nodes[startKey] ]

    endCoord ← graph.nodes[endKey]
    if endCoord = null then
        return null

    // — Initialisation



---


    gScore ← empty map    // g(n): actual cost from start → n
    fScore ← empty map    // f(n) = g(n) + h(n)
    cameFrom ← empty map  // for path reconstruction

    gScore[startKey] ← 0

    startCoord ← graph.nodes[startKey]
    h0 ← HAVERSINE(startCoord.lat, startCoord.lng,
                    endCoord.lat, endCoord.lng)
    fScore[startKey] ← h0

    openSet ← min-priority-queue keyed on fScore
    openSetKeys ← empty set    // for fast membership test

```

```

openSet.push( (fScore[startKey], startKey) )
openSetKeys.add(startKey)

// — Main loop

```

```

while openSet is not empty do

    // 1. Dequeue node with lowest f-score
    (_, current) ← openSet.pop_min()
    openSetKeys.remove(current)

    // 2. Goal check
    if current = endKey then
        return RECONSTRUCT_PATH(cameFrom, graph.nodes, current)

    // 3. Expand neighbours
    currentG ← gScore[current] // always defined at this point

    for each edge (neighbourKey, weight, coord) in graph.adjacency[current] do

        tentativeG ← currentG + weight

        if tentativeG < gScore.get(neighbourKey, default=∞) then

            // Better path found — update records
            cameFrom[neighbourKey] ← current
            gScore[neighbourKey] ← tentativeG
            h ← HAVERSINE(coord.lat, coord.lng,
                          endCoord.lat, endCoord.lng)
            fScore[neighbourKey] ← tentativeG + h

            // Add to open set if not already present
            if neighbourKey ∉ openSetKeys then
                openSet.push( (fScore[neighbourKey], neighbourKey) )
                openSetKeys.add(neighbourKey)

    // — No path exists

```

```

return null
function RECONSTRUCT_PATH(cameFrom, nodes, current) → list of LatLng

path ← empty list
node ← current

```

```

while node ≠ undefined do
    path.prepend( nodes[node] ) // add coordinate to front
    node ← cameFrom.get(node) // follow parent pointers

return path

```

Figure 3.4. A* Algorithm PseudoCode

3.3.4. End-to-End Data Flow

When a user initiates a route, the following sequence takes place: 1. Origin resolution: The starting point is determined either from the device's live GPS position or from a building the user has manually selected. 2. Snap to graph: Both the origin and destination coordinates are mapped to their nearest nodes on the campus road network, ensuring the route begins and ends on a valid road segment. 3. Route computation: The routing engine calculates the shortest path between the two snapped nodes and returns an ordered list of geographic waypoints. 4. Map rendering: The computed waypoints are drawn on the map as a visible route path. 5. Auto-zoom: The map camera automatically adjusts its zoom level and position to frame the entire route within the screen, accounting for the bottom control panel.

3.4. User Interface (UI) Design

3.4.1. Design Principles

The UI design follows a **dark-mode navigation aesthetic**, inspired by the UI patterns of Google Maps Night Mode, Apple Maps, and Waze. The design priorities were:

- **Map visibility:** The map must dominate the screen. Overlaid UI elements are minimal, semi-transparent where possible, and positioned at screen edges.
- **Single-thumb reachability:** The route planner controls are placed at the bottom of the screen, within comfortable reach of a user's thumb when holding a phone in one hand.
- **Status clarity:** The current route state (no route / route active) must be immediately legible at a glance.

3.4.2. Main Map Screen

The main screen is composed of the following layers:

- **MapView (Full-screen):** Google Maps with `mapType="standard"`, overlaid with:
 - A semi-transparent blue-grey Polygon tracing the campus boundary.
 - Custom Marker badges for each of the 12 landmarks (white pill with blue border; active destination highlighted in solid blue).
 - The dual-layer route Polyline (white stroke, width 9, beneath the blue stroke, width 5) for a Google Maps navigation visual style.
 - A "You are here" dot (pulsing blue circle) at the user's GPS position.
- **Vignette gradients:** Four `<LinearGradient>` overlays (top, bottom, left, right edges) provide a subtle darkening of the map perimeter, focusing the user's visual attention on the campus centre.

3.4.3. Route Panel (Bottom Sheet)

The route planner is implemented as a draggable bottom sheet panel that can be pulled up or pushed down by the user. It snaps smoothly between two fixed heights — an expanded state showing the full route controls, and a collapsed state showing only the panel title. The transition between states uses a spring animation, and the panel content fades in and out accordingly as the height changes.

3.4.4. Building Picker Modal

The building picker is presented as a full-screen modal that slides up from the bottom of the screen. It contains a real-time search bar that instantly filters the list of campus landmarks as the user types. A 'My Location' option is available at the top of the list when selecting the starting point, allowing users to navigate from their current GPS position. Each entry in the list displays the building's name and a short description, and tapping any item immediately sets it as the selected location and dismisses the modal.

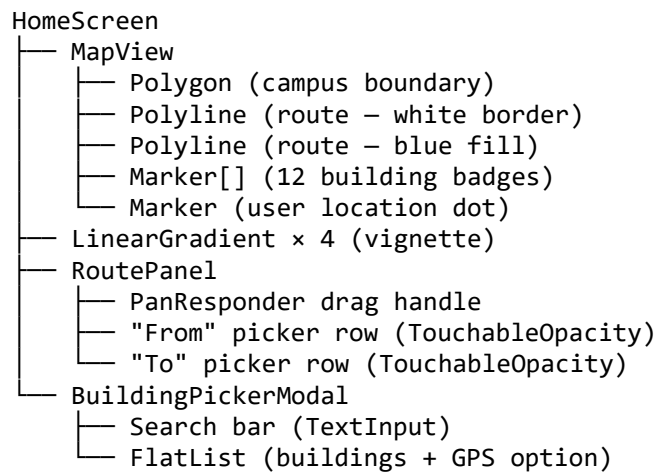


Figure 3.5. UI Component Hierarchy

CHAPTER 4. RESULTS AND DISCUSSION

Because this is a demo and not a final release, it still has several limitations regarding user experience optimization, performance management, and overall refinement.

4.1. Implementation Environment

Development and testing were conducted in the following environment:

Table 4.1. Development Environment Specifications

Component	Specification
Operating System	Windows 11 (development host)
IDE	Visual Studio Code with Expo Tools extension
Node.js	v20.x LTS
npm	v11.6.1
Expo SDK	~54.0.0 (React Native 0.81.5)
TypeScript	~6.0.3
react-native-maps	1.20.1
expo-location	~19.0.8
expo-linear-gradient	~15.0.8
expo-router	~6.0.24
Test Device (Android)	Mid-range Android smartphone (Android 13)
Map Provider	Google Maps SDK (PROVIDER_GOOGLE)

The project follows the **Expo managed workflow** (no native code ejection required). The development server is started with ``npm run expo start``, and the Expo Go client application on the test device connects via LAN to receive the JavaScript bundle over-the-air.

4.2. Application Interfaces and Functionalities

4.2.1. Application Launch

When the application is first opened, it automatically requests permission to access the device's location through the standard system dialog. If the user grants this permission, the application begins tracking the GPS position in real time, updating the location indicator whenever the user moves or after a short interval.

The main screen then loads with the EIU campus placed centrally in view. The campus boundary is immediately visible as a semi-transparent overlay, allowing the user to clearly distinguish the campus area from the surrounding map.



4.2.2. Route Planning — Picker Flow

The user expands the Route Planner panel by either tapping the handle pill or dragging upward. Tapping the **FROM** row opens the `BuildingPickerModal` with the "My Location" (GPS) option shown at the top. Tapping the **TO** row opens the same modal without the GPS option. Within the modal, the user can type in the search bar to filter buildings by name (e.g., typing "b8" narrows the list to "Block 8").

Upon selection, the modal dismisses with a slide-down animation, the selected value populates the corresponding picker row, and `computeRoute()` is immediately invoked if both endpoints are defined.

Route Planner



FROM

My Location

Current GPS position

>



TO

Select destination...

>

Search buildings...

**My Location**
Use current GPS position

>

**Block 3**
B3

>

**Block 8**
B8

>

**Block 9**
B9

>

**Block 10**
B10

>

**Block 11 (IT)**
B11

>

**Canteen**
CANTEEN

>

**Library**
LIBRARY

>

**Recruitment Office**
recruit

>

**Parking Zone 1**
PARKING

>

Search buildings...

**Block 3**
B3

>

**Block 8**
B8

>

**Block 9**
B9

>

**Block 10**
B10

>

**Block 11 (IT)**
B11

>

**Canteen**
CANTEEN

>

**Library**
LIBRARY

>

**Recruitment Office**
recruit

>

**Parking Zone 1**
PARKING

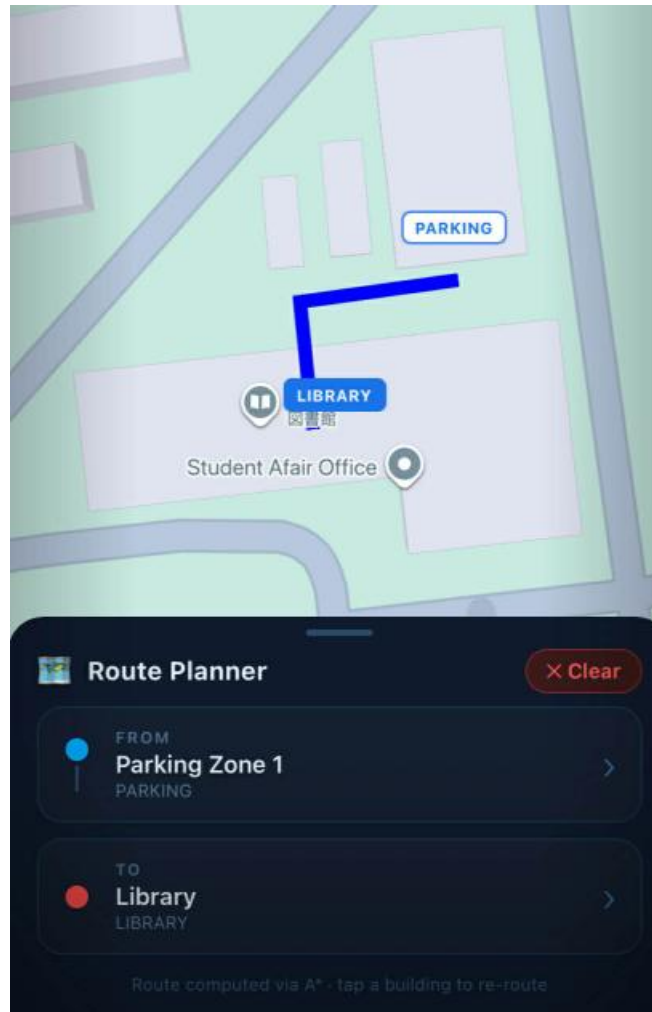
>

**Parking Zone 2**
PARKING2

>

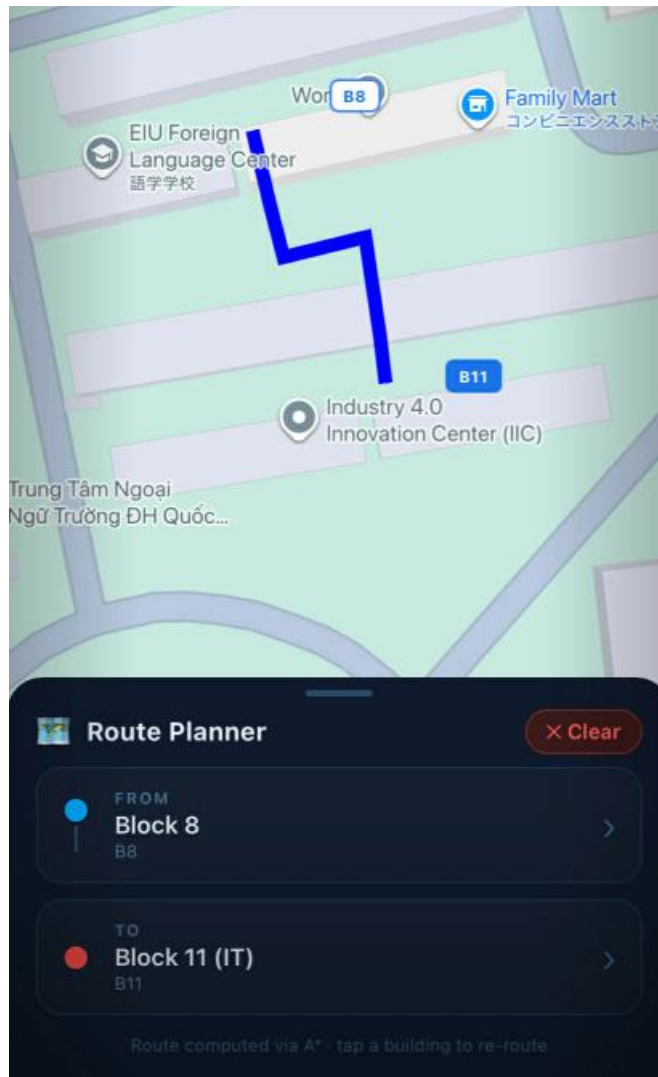
4.2.3. Route Planning — Marker Tap Flow

As an alternative to the picker modal, the user can tap any building marker badge directly on the map. This sets the tapped building as the **destination**, uses the current GPS position (or previously selected origin) as the **start**, and computes the route immediately. The map then animates to the tapped building's location and displays the building's callout (name + description) after a short delay.



4.2.4. Route Display

Once calculated, the route is visually presented on the map as a clear blue path. To provide immediate context, the map automatically adjusts its zoom level and pans smoothly to frame the entire journey. The viewport is also padded to ensure that no part of the route is hidden behind the bottom control panel or screen edges.



4.2.5. Route Clearing

Whenever an active route is displayed on the map, a prominent 'Clear' button becomes available in the route planner panel. Tapping this button provides users with a quick and easy way to cancel their current navigation. It instantly removes the drawn path from the map and resets the selected locations, allowing the user to seamlessly start searching for a new destination.

 **Route Planner** ✕ Clear


|

FROM
Block 8
B8

>



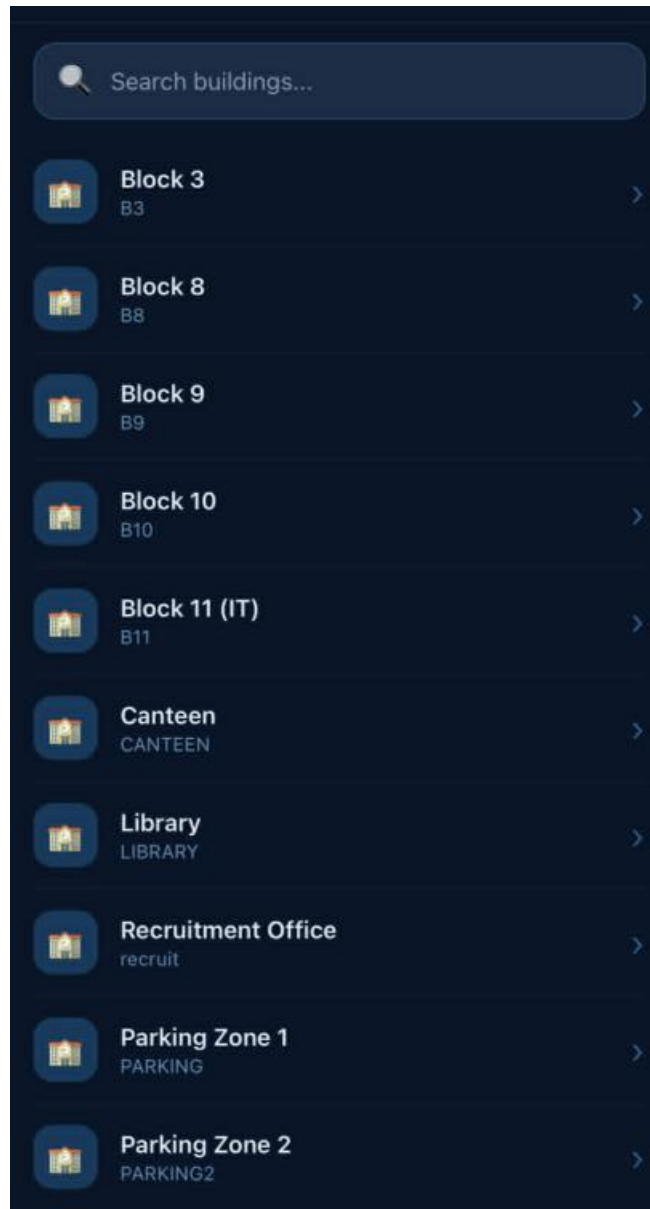
TO
Block 11 (IT)
B11

>

Route computed via A* - tap a building to re-route

4.2.6. Building Listing Panel

The application includes a scrollable directory panel listing all campus landmarks and their coordinates. This provides a convenient reference guide for users to quickly explore the university layout.



4.3. Demo Observations and Limitations.

4.3.1. Routing Correctness

During the demo, selected routes between campus landmarks were tested manually. The computed paths correctly followed the actual road network without cutting through buildings. Observations confirmed that the A* algorithm produced reasonable and visually accurate results for all tested cases.

4.3.2. Performance

Route computation was noticeably fast across all tested cases, with no visible delay from the user's perspective. Given the small scale of the campus road network, the algorithm performed adequately for the demo's purpose.

4.3.3. Limitation

One observed limitation during testing was occasional GPS inaccuracy near building facades, where signal interference may cause slight positional shifts. This is a known constraint of GNSS technology in dense environments and represents an area for improvement in future versions.

4.3.4. UI/UX Assessment

The overall interface was found to be straightforward during the demo phase. The draggable route panel and building search modal were easy to interact with. It was also noted that building markers can become visually crowded at lower zoom levels, which is a known limitation to be addressed in future iterations.

CHAPTER 5. CONCLUSION AND FUTURE WORKS

5.1. Conclusion

This project successfully demonstrated the core objectives through a functional prototype of a mobile campus navigation application for Eastern International University. The following outcomes were achieved during the demo phase:

Spatial Data Integration: EIU's GIS Shapefile data was successfully converted to GeoJSON format and embedded in the application as static assets, enabling fully offline routing without backend infrastructure.

Graph-Based Routing Engine: A custom A* pathfinding implementation over a campus road graph was able to produce reasonable walking routes between named landmarks with no noticeable delay during testing.

GPS Localisation: Real-time GPS tracking via expo-location was integrated to resolve the user's current position and enable "navigate from here" functionality.

User Interface: A dark-themed interface with a draggable bottom sheet, custom building badges, and route polylines was implemented, providing a clean and functional navigation experience for the demo.

Cross-Platform Deployment: The single TypeScript codebase targets both iOS and Android through the React Native/Expo managed workflow.

As this is a demonstration prototype, the application still has room for improvement in areas such as performance optimisation, user experience refinement, and extended feature coverage. These limitations represent natural directions for future development

5.2. Future Works

Several directions for extending are identified:

1. **Zoom-Dependent Marker Clustering:** At low zoom levels, overlapping building markers should be clustered into a single aggregate marker showing the count. At high zoom levels, individual markers should expand. Libraries such as ``react-native-maps-super-cluster`` support this pattern.

2. **Indoor Floor-Level Navigation:** The current system navigates between buildings but not within them. Extension to multi-storey indoor navigation could be achieved by integrating Wi-Fi fingerprinting positioning (using the device's Wi-Fi scan results against a pre-recorded fingerprint database) or BLE beacon triangulation, combined with floor-specific GeoJSON overlays for each building level.

3. **Real-Time Campus Events Integration:** A backend API could push location-tagged events (e.g., exam schedules, administrative notices) that appear as time-limited markers on the map.

4. **Pedestrian Traffic Heatmaps:** Anonymised, aggregated GPS telemetry from opted-in users could generate heatmaps indicating congestion on campus pathways at different times of day, enabling the routing engine to prefer less-congested routes.

REFERENCES

1. P. E. Hart, N. J. Nilsson, and B. Raphael, "A Formal Basis for the Heuristic Determination of Minimum Cost Paths," **IEEE Transactions on Systems Science and Cybernetics**, vol. 4, no. 2, pp. 100–107, 1968.
2. W. G. Griswold, R. Boyer, S. W. Brown, and T. M. Truong, "Using Mobile Technology to Create Opportunistic Interactions on a University Campus," **Advances in Pervasive Computing: Workshop Proceedings**, 2004.
3. T. Mirowski, S. Whiting, H. Steck, R. Palaniappan, M. MacDonald, D. Hartl, and T. S. Ho, "Probability Kernel Regression for WiFi Localisation," **Journal of Ambient Intelligence and Smart Environments**, vol. 5, no. 4, pp. 407–418, 2013.
4. GeoJSON Specification, "The GeoJSON Format," **RFC 7946**, Internet Engineering Task Force (IETF), August 2016. [Online]. Available: <https://www.rfc-editor.org/rfc/rfc7946>
5. Expo Documentation, "expo-location," Version 19.0.0, Expo SDK v54, 2025. [Online]. Available: <https://docs.expo.dev/versions/v54.0.0/sdk/location/>
6. React Native Community, "react-native-maps," GitHub Repository, 2024. [Online]. Available: <https://github.com/react-native-maps/react-native-maps>
7. Movable Type Scripts, "Haversine Formula — Calculate Distance, Bearing and More Between Latitude/Longitude Points," 2023. [Online]. Available: <https://www.movable-type.co.uk/scripts/latlong.html>
8. ESRI, "Shapefile Technical Description: An ESRI White Paper," Environmental Systems Research Institute, Redlands, CA, July 1998.
9. Meta Platforms, "React Native Documentation," Version 0.81, 2025. [Online]. Available: <https://reactnative.dev/docs/getting-started>
10. S. Russell and P. Norvig, *Artificial Intelligence: A Modern Approach*, 4th ed. Hoboken, NJ: Pearson, 2020.